C:\hpc-23-feb-2026\.\34\eg212.cpp

```
#include<iostream>
using namespace std;
class input_work
{
private:
int num;
public:
input_work(int num)
{
this->num=num;
}
int operator()()
{
return this->num;
}
};
class processing_work
{
public:
int operator()(int e)
{
return e*e;
}
};
template<typename T1,typename T2>
T1 operator &(T1 left,T2 right)
{
int data=left(); // T1 is functor, left is callable object
return T1(right(data));
}
class output_work
{
public:
int operator()(int num)
{
return num;
}
};
void do_task(input_work w)
{
cout<<w()<<endl;
}
int main()
{
input_work input_provider(10);
processing_work processor;
output_work output_generator;
do_task(input_provider & processor & output_generator);
```

```
// 32 & 55 & 66  example of bitwise and
// x & y     example of bitwise and
// &x     not as parameter, address operator
// int &x   reference operator for creating alias
// && logical
// In the above example & will be treated as bitwise operator
// for this to work & should be overloaded as bitwise operator
return 0;
}
```

Thinking Machines Classroom Codes